

Injection Function and Constructor injection

- In the angular 14 we can use the injection function instead of Construction injection

1- Import injection from Angular core

```
import { Component, inject } from '@angular/core';
```

```
private counterService : CounterService = inject(CounterService);
```

```
TS counter.ts TS counter-display.ts X
src > app > counter-display > TS counter-display.ts > ...
1  import { Component, inject } from '@angular/core';
2  import { CounterService } from '../services/counter';
3
4  @Component({
5    selector: 'app-counter-display',
6    imports: [],
7    templateUrl: './counter-display.html',
8    styleUrls: ['./counter-display.css'],
9  })
10 export class CounterDisplay {
11
12   private counterService : CounterService = inject(CounterService);
13
14   // constructor(private counterService: CounterService){}
15
16   getCurrentCount(): number{
17     return this.counterService.getCount();
18   }
19
20 }
21
```

inject Function vs Constructor Injection

Why would you use inject()?

Benefit #1 – Cleaner code

- ✓ No constructor needed (unless extra init logic)
- ✓ Dependencies declared as class properties
- ✓ Less boilerplate → easier to read

constructor
- property = inject(...)



Benefit #2 – More flexible usage

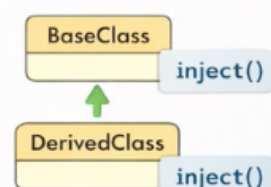
- ✓ Can call inside class methods
- ✓ Can call inside arrow functions
- ✓ Can call in standalone functions

method() () → inject()

(→()) function inject()

Benefit #3 – Plays nicely with inheritance

- ✓ Base class + derived class both need services
- ✓ No “super(...)” parameter passing hassle



inject Function vs Constructor Injection

Which one should you use? (Recommendation)

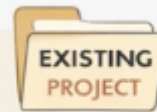


New Project / New Component

✓ Use **inject()**

- Modern approach (Angular 14+)
- Cleaner code
- More flexibility

```
private counterService = inject(  
  CounterService);
```



Existing Codebase

✓ Keep **constructor injection**

- No need to refactor
- Works identically
- Prefer consistency / team preference

```
constructor(private counterService:  
  CounterService) {
```

WEB ACADEMY
BY HARSHA VARDHAN



Both approaches can coexist in the same app

- In this course: you'll see **both**
- Task Manager project: **inject ()** used more often

inject Function vs Constructor Injection

Wrap-up & What's Next

Key Takeaways

- ✓ **inject** is a modern alternative to constructor injection
- ✓ Both approaches do the same thing
- ✓ **inject** = cleaner syntax + more flexibility

You now understand...

- ☐ **Services** (what they are)
- ☐ Creating services
- ☐ Dependency Injection (DI)
- ☐ Sharing data between components
- ☐ Injector hierarchy (levels of providers)